UNIVERSITAS
GADJAH MADA

LABSHEET

Workshop Implementasi Agentic AI untuk Oil & Gas Industry

Bab Bonus: Membangun Agen SQL

Pusat Kajian LKFT
Fakultas Teknik, Universitas Gadjah Mada
2026

Daftar Isi

Prasyarat 3

Tujuan Pembelajaran 3

Analogi: Mencari Buku di Perpustakaan Besar 3

Mengapa Agen SQL Perlu Kehati-hatian Ekstra 5

Database Contoh: migas.db 6

Membangun Agen SQL 6

- Contoh 1: Agen SQL yang Naif 6
- Contoh 2: Biarkan Agen Menjelajahi Skema 10
- Contoh 3: Membuat SQL Aman 16
- Contoh 4: Error adalah Informasi 20
- Contoh 5: Agen SQL yang Aman (Menggabungkan Semuanya) 23

Pilihan Rancangan yang Perlu Diketahui 29

Poin Penting 30

Peta Konsep 31

Latihan 31

Sumber Tambahan 32

Apa Selanjutnya? 32

Pemecahan Masalah 32

Dependency 34

Referensi 34

Kode Jawaban Latihan 35

Membangun Agen SQL

Di bab ini Anda membangun agen yang **menjawab pertanyaan dari database**. Anda mulai dengan agen naif yang menebak nama tabel, memberinya tool untuk menjelajahi skema, membungkusnya dengan lapisan perlindungan agar ia hanya bisa *membaca* data, dan akhirnya menggabungkan semua yang sudah dipelajari (jawaban bertipe, redaction data pribadi, persetujuan manusia, dan batas panggilan) menjadi satu agen SQL yang aman dan siap produksi.

> **Bab bonus.** Bab ini opsional. Bab-bab berikutnya tidak bergantung padanya, jadi boleh dilewati atau dikerjakan di akhir jika waktu tersisa.

Prasyarat

- Sudah menyelesaikan Guardrails dan Middleware Keamanan (lapisan keamanan) dan Persetujuan Manusia
- Sudah menyelesaikan Structured Output pada Agen dan Memori Agen dan Checkpointer
- SQL dasar: Anda harus bisa membaca `SELECT ... FROM ... JOIN ... GROUP BY`. Anda tidak perlu menulisnya; agenlah yang menulis

Tujuan Pembelajaran

Setelah bab ini Anda bisa:

- Menjelaskan apa itu agen text-to-SQL dan di mana ia bisa keliru
- Memberi agen tool untuk mendaftar tabel serta memeriksa kolom dan baris contoh
- Menjalankan SQL dengan aman: pemeriksaan hanya-SELECT, koneksi hanya-baca, batas baris, dan timeout
- Mengembalikan pesan error yang membantu agar agen bisa memperbaiki kesalahannya sendiri
- Menerapkan redaction pada data pribadi di hasil query sebelum model melihatnya
- Memasang persetujuan manusia di depan apa pun yang menulis file
- Menguji agen SQL terhadap kebenaran yang dihitung secara independen
- Menerapkan pola yang sama pada database lain

Analogi: Mencari Buku di Perpustakaan Besar

Bayangkan dua orang masuk ke perpustakaan besar di hari pertama. Keduanya diminta: "Carikan semua buku tentang kucing."

Si pemula masuk ke ruang buku dan berteriak: *"Mana buku tentang kucing?"* Tidak ada rak bernama begitu. Setelah beberapa tebakan salah, seseorang memberi tahu bahwa buku-buku itu disimpan di rak dengan kode `Hewan/Peliharaan`. Lambat, berisik, dan beberapa buku yang salah ikut terbawa.

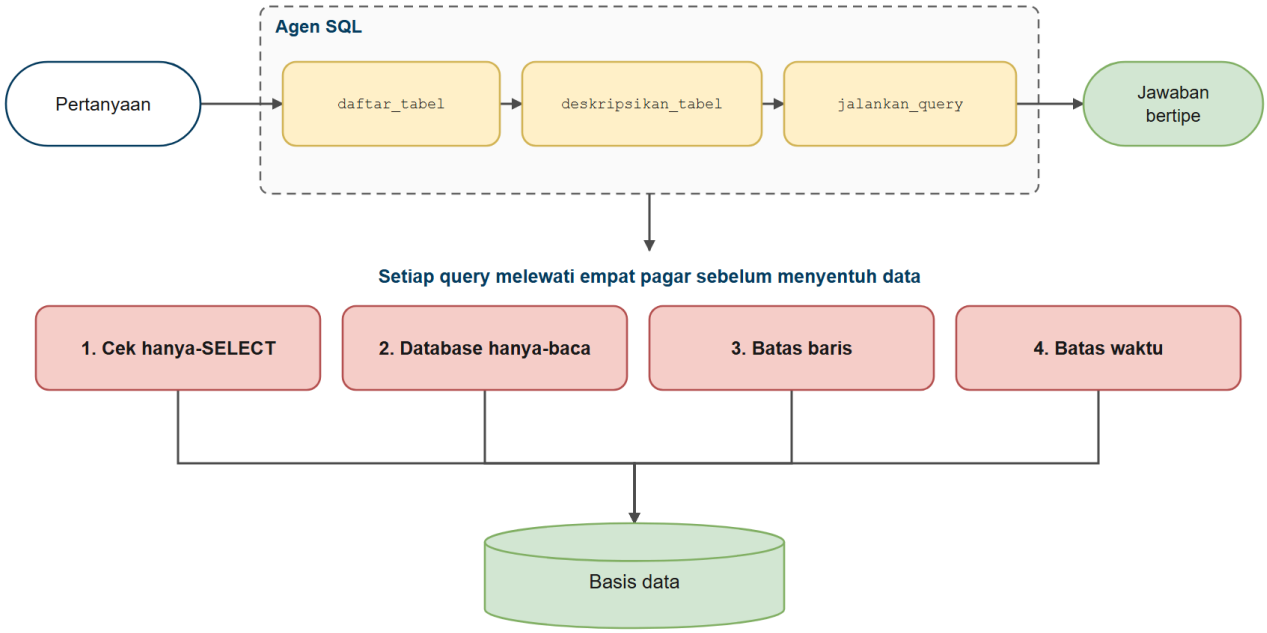
Si profesional melakukan ini:

1. Membaca **katalog** untuk tahu rak apa saja yang ada *(daftar tabel)*
2. Membuka rak yang tepat dan melihat sekilas **label kolom dan beberapa buku contoh** *(deskripsikan tabel)*
3. Baru kemudian meminta **persis** buku yang dibutuhkan *(jalankan query)*
4. Memeriksa hasilnya, dan kalau terlihat aneh, kembali dan memeriksa ulang *(pemulihan error)*

Perpustakaan itu juga punya aturan: **buku boleh dibaca tetapi tidak boleh dicoret atau disobek**, dan memfotokopi buku untuk dibawa pulang butuh tanda tangan petugas.

Agen SQL bekerja dengan cara yang sama.

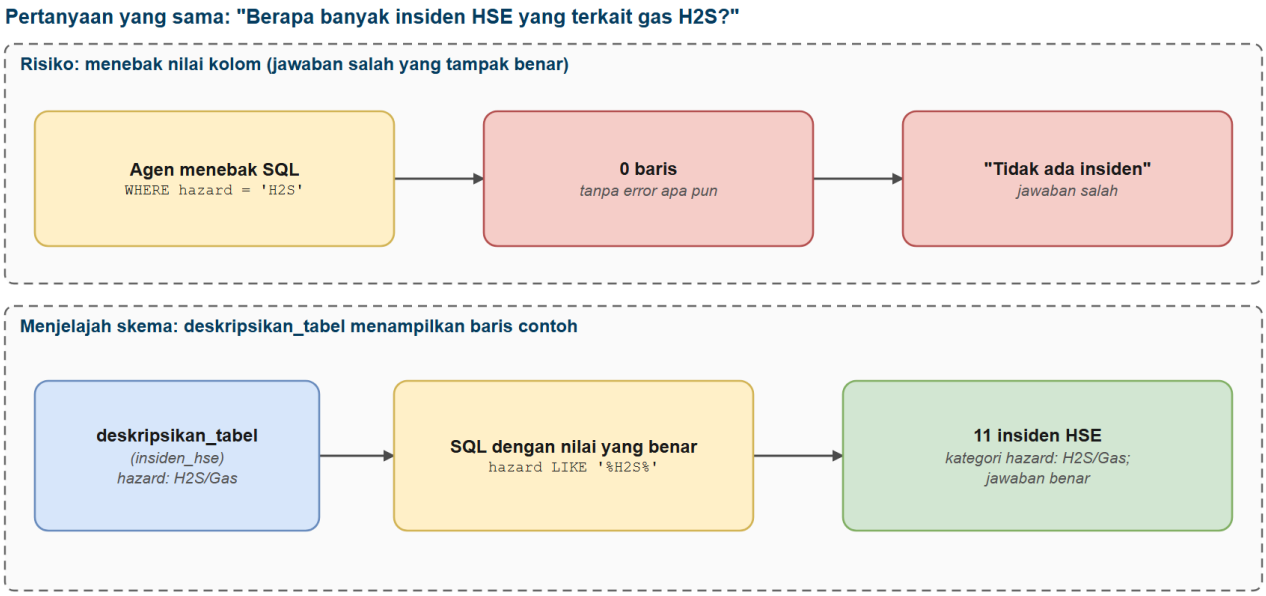
Di perpustakaan	Di agen SQL
Katalog	`daftar_tabel`
Label kolom + buku contoh	`deskripsikan_tabel` (kolom + baris contoh)
Meminta buku tertentu	`jalankan_query` (satu `SELECT`)
"Hanya dibaca: dilarang mencoret"	Koneksi hanya-baca + pemeriksaan hanya-`SELECT`
Nama peminjam disembunyikan	PII redaction pada hasil tool
Tanda tangan petugas untuk memfotokopi	Persetujuan manusia pada tool ekspor



Gambar 1. Query agen melewati empat pagar sebelum ke basis data

Mengapa Agen SQL Perlu Kehati-hatian Ekstra

Agen SQL mengubah sebuah *kalimat* menjadi *kode yang berjalan di atas data Anda*. Itu kuat, dan berisiko.



Gambar 2. Menebak nilai kolom vs menjelajah skema

Risiko	Contoh	Pertahanan (bab ini)
Jawaban salah yang tampak benar	`WHERE hazard = 'H2S'` mengembalikan 0 baris, padahal datanya tertulis `'H2S/Gas'`	Penjelajahan skema dengan baris contoh (Contoh 2)
SQL yang merusak	`DROP TABLE work_order`	Koneksi hanya-baca + pemeriksaan hanya-SELECT (Contoh 3)
Query yang lepas kendali	`JOIN` berlapis yang tidak pernah selesai	Timeout dan batas baris (Contoh 3)
Kebocoran data	Nama pelapor muncul di jawaban	PII redaction pada hasil tool (Contoh 5)
Efek samping	Menulis file, memanggil sistem lain	Persetujuan manusia (Contoh 5)
Retry tanpa akhir	Agen berputar pada query yang rusak	Batas panggilan model (Contoh 4-5)

> **Aturan emas:** jangan pernah mengandalkan prompt ("hanya jalankan SELECT") untuk menjaga SQL tetap aman. Pasang di **kode**, dan idealnya di **izin database** juga.

Database Contoh: migas.db

Kita memakai `migas.db`, database SQLite kecil yang dibangun dari data pelatihan lapangan fiktif (work order pemeliharaan dan laporan insiden HSE). Database ini dibangun oleh fungsi `buat_database_migas()` dari paket bersama dan selalu berisi data yang sama (SQLite, tanpa server).

Tabel	Baris	Isinya
`work_order`	400	Work order pemeliharaan: tanggal, area (Seruni, Kenanga, Melati), aset, jenis peralatan, kategori kegagalan, tindakan, `downtime_hours`, `safety_flag`, dan catatan asli teks bebas
`insiden_hse`	120	Laporan insiden: tanggal, lokasi, narasi asli, `incident_type`, `hazard`, `severity`

> Semua data fiktif untuk pelatihan. Kolom `dilaporkan_oleh` pada `work_order` berisi nama depan pelapor; itulah data pribadi yang akan kita lindungi di Contoh 5.

Tool yang bisa dipakai ulang ada di `code/sql_tools.py`. Contoh 2-5 mengimpornya; Contoh 1 mendefinisikan tool-nya langsung agar Anda melihat rancangannya berkembang.

> **Memakai notebook?** Contoh mengimpor `sql_tools` dari folder `code`. Saat Anda menjalankan file `.py`, itu otomatis; di notebook, jalankan sel ini sekali lebih dulu:

```
import sys
from pathlib import Path

# Buat sql_tools.py dapat diimpor (berlaku baik notebook dijalankan dari akar repo maupun dari hari4/bonus-agen-sql/)
folder_code = Path("hari4/bonus-agen-sql/code") if Path("hari4/bonus-agen-sql/code").exists() else Path("code")
sys.path.insert(0, str(folder_code.resolve()))
```

Membangun Agen SQL

Contoh 1: Agen SQL yang Naif

Agen SQL paling sederhana punya **satu tool**: `jalankan_query(sql)`. Model harus menulis SQL untuk database yang belum pernah ia lihat.

Kode: `code/01_naive_sql_tool.py` **Jalankan**: `python hari4/bonus-agen-sql/code/01_naive_sql_tool.py`

```
# %% [markdown]
# # Contoh 1: Agen SQL yang Naif
```

```python
# Jalankan sel satu per satu dengan Shift+Enter, atau seluruh file: `python hari4/bonus-agen-sql/code/01_naive_sql_tool.py`
#
# Agen SQL paling sederhana punya SATU tool: jalankan_query(sql). Model harus menulis SQL untuk database
# yang belum pernah ia lihat. Perhatikan apa yang terjadi: ia menebak nama tabel dan kolom, dan database
# membalas dengan error.
#
# (Koneksinya hanya-baca, sehingga percobaan ini tidak bisa merusak data. Perlindungan yang Anda tambahkan
# nanti di bab ini jauh lebih penting ketika koneksinya BUKAN hanya-baca.)

# %%
# Langkah 1 - Menyiapkan model dan database
import os
import sqlite3
from contextlib import closing
from pathlib import Path

from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain_core.messages import AIMessage, HumanMessage
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI

from migas_shared.data import DB_DEFAULT, buat_database_migas

load_dotenv()

DB_PATH = Path(DB_DEFAULT)
if not DB_PATH.exists():
    buat_database_migas(DB_PATH)  # dibangun dari CSV pelatihan, isinya selalu sama

model = ChatOpenAI(
    model=os.getenv("ONLINE_MODEL"),
    base_url=os.getenv("ONLINE_BASE_URL"),
    api_key=os.getenv("ONLINE_API_KEY"),
    reasoning_effort=os.getenv("LLM_REASONING_EFFORT", "none"),
)

# %% [markdown]
# ## Langkah 2 - Satu tool SQL dan agennya
# Tool `jalankan_query` menerima SQL apa saja dan mengembalikan hingga 20 baris. Agen tidak diberi tool
# untuk melihat skema.

# %%
@tool
def jalankan_query(sql: str) -> str:
    """Jalankan query SQL pada basis data dan kembalikan hasilnya.

    Args:
        sql: Query SQL yang dijalankan
    """
    try:
        with closing(sqlite3.connect(f"file:{DB_PATH.resolve().as_posix()}?mode=ro", uri=True)) as sambungan:
            baris = sambungan.execute(sql).fetchmany(20)
        return str(baris)
    except sqlite3.Error as error:
        return f"Error SQL: {error}"
```

```
agent = create_agent(
    model,
    tools=[jalankan_query],
    system_prompt="Anda menjawab pertanyaan tentang basis data operasi migas dengan SQL. Jawab
singkat.",
)

# %% [markdown]
# ## Langkah 3 - Menanyakan agen
# Pertanyaan ini mensyaratkan pengetahuan tentang skema yang tidak dimiliki agen.

# %%
pertanyaan = "Berapa banyak insiden HSE yang bahayanya terkait gas H2S?"
print(f"Pengguna: {pertanyaan}\n")
response = agent.invoke({"messages": [HumanMessage(content=pertanyaan)]})

# %% [markdown]
# ## Langkah 4 - Melihat SQL yang dicoba agen
# `tampilkan_query` mencetak setiap query beserta statusnya dan menghitung percobaan yang
gagal.

# %%
def tampilkan_query(response) -> tuple[int, int]:
    """Cetak setiap query SQL yang dicoba agen. Mengembalikan (percobaan, percobaan_gagal)."""
    percobaan = 0
    gagal = 0
    pesan_semua = response["messages"]
    for indeks, pesan in enumerate(pesan_semua):
        if isinstance(pesan, AIMessage):
            for panggilan in pesan.tool_calls:
                percobaan += 1
                hasil = next(
                    (m.content for m in pesan_semua[indeks + 1:] if getattr(m, "tool_call_id",
None) == panggilan["id"]),
                    "",
                )
                status = "GAGAL" if str(hasil).startswith("Error SQL") else "OK   "
                gagal += status == "GAGAL"
                print(f"   {status} Percobaan {percobaan}: {panggilan['args']['sql']}")
                if status == "GAGAL":
                    print(f"         -> {hasil}")
    return percobaan, gagal


print("SQL yang dicoba agen:")
percobaan, gagal = tampilkan_query(response)
print(f"\nAgen: {response['messages'][-1].content}\n")

# %% [markdown]
# ## Langkah 5 - Yang baru saja terjadi

# %%
print("Yang baru saja terjadi:")
print(f"   - Agen memakai {percobaan} panggilan query; {gagal} di antaranya gagal")

# %% [markdown]
# ## Ide Utama
# - Agen harus MENEBAK nama tabel dan kolom sebelum bisa bertanya apa pun
# - Setiap tebakan salah adalah panggilan model tambahan: lebih lambat dan lebih mahal
# - Model yang cerdik mungkin pulih dengan mengintip `sqlite_master`, tetapi tidak ada jaminan
# - Lebih buruk lagi, tebakan salah bisa tampak benar: `WHERE hazard = 'H2S'` menghasilkan 0
baris tanpa
```

```
#   error apa pun (datanya tertulis `H2S/Gas`)
# - Berikutnya: beri agen tool untuk MENJELAJAHI database sebelum bertanya
```

Keluaran yang Diharapkan

> **Catatan:** jawaban model bisa berbeda setiap kali dijalankan. Keluaran di seluruh bab ini berasal dari menjalankan skrip sebenarnya. Pada contoh ini jumlah percobaan memang bervariasi menurut model; variasi itulah intinya.

```
Pengguna: Berapa banyak insiden HSE yang bahayanya terkait gas H2S?

SQL yang dicoba agen:
   GAGAL Percobaan 1: SELECT COUNT(*) AS jumlah_insiden_h2s
FROM hse_incidents
WHERE LOWER(bahaya) LIKE '%h2s%' OR LOWER(bahaya) LIKE '%hydrogen sulfide%' OR LOWER(bahaya)
LIKE '%gas beracun%'
         -> Error SQL: no such table: hse_incidents
   OK    Percobaan 2: SELECT name FROM sqlite_master WHERE type='table'
   OK    Percobaan 3: PRAGMA table_info(insiden_hse)
   OK    Percobaan 4: SELECT COUNT(*) AS jumlah_insiden_h2s
FROM insiden_hse
WHERE LOWER(hazard) LIKE '%h2s%' OR LOWER(hazard) LIKE '%hydrogen sulfide%'

Agen: Berikut hasilnya:

**Jumlah insiden HSE yang bahayanya terkait gas H2S adalah 11 insiden.**

Yang baru saja terjadi:
   - Agen memakai 4 panggilan query; 1 di antaranya gagal
```

Cara Kerjanya

1. **Model mulai dalam keadaan buta.** Ia menebak nama tabel dan kolom berdasarkan kebiasaan penamaan, padahal nama sebenarnya adalah `work_order`, `insiden_hse`, `hazard`, dan seterusnya.
2. **Error kembali sebagai teks.** Karena tool *mengembalikan* `"Error SQL: ..."` dan tidak melempar exception, model bisa membacanya dan mencoba lagi. Itulah satu-satunya alasan agen ini bisa pulih.
3. **Tebakan yang salah bisa tampak benar.** `WHERE hazard = 'H2S'` adalah SQL yang valid. Ia tidak gagal, ia hanya mengembalikan **0**. Model yang kurang teliti bisa menjawab "tidak ada insiden". Hanya dengan memeriksa nilai sebenarnya orang tahu bahwa datanya tertulis `'H2S/Gas'`.
4. **Setiap tebakan salah adalah panggilan model tambahan**: lebih lambat dan lebih mahal.
5. **Model yang cerdik sering selamat, tetapi itu keberuntungan.** Model modern sering memakai `LIKE '%H2S%'` atau mengintip skema sendiri, dan menjawab dengan benar (seperti pada keluaran di atas). Pada eksekusi lain atau dengan model lain, ia bisa saja memakai `=` dengan nilai yang salah dan menjawab 0 tanpa curiga. Jangan membangun di atas keberuntungan.

> **Pelajaran:** agen SQL yang tidak bisa *melihat* database akan menebak. Sebagian model menebak dengan cerdik, tetapi Anda tidak boleh membangun di atas keberuntungan.

Contoh 2: Biarkan Agen Menjelajahi Skema

Obat untuk menebak adalah melihat. Kita menambahkan dua tool dan system prompt yang menyuruh memakainya lebih dulu.

Tool	Yang dikembalikan	Mengapa membantu
`daftar_tabel()`	Semua nama tabel	"Indeks" arsip
`deskripsikan_tabel(tabel)`	Kolom beserta tipe **dan 3 baris contoh**	Menunjukkan nama persis *dan* bentuk nilai (`'H2S/Gas'`, format tanggal...)
`jalankan_query(sql)`	Baris dari satu `SELECT`	Pertanyaan sebenarnya

Kode: `code/02_schema_tools.py` (tool ada di `code/sql_tools.py`) **Jalankan**: `python hari4/bonus-agen-sql/code/02_schema_tools.py`

```
"""
Modul bantu bab bonus Agen SQL: tool SQL yang aman untuk agen.

Dipakai oleh Contoh 2-5 dan solusi latihan. Contoh 1 mendefinisikan tool-nya langsung di file
contoh
agar Anda melihat rancangannya berkembang; file inilah tempat rancangan itu berakhir.

Modul ini menyediakan tiga tool dan lapisan perlindungan di sekelilingnya:

  daftar_tabel     : tabel apa saja yang ada
  deskripsikan_tabel : kolom + beberapa baris contoh dari satu tabel
  jalankan_query   : jalankan SATU SELECT hanya-baca, dengan batas baris

  Lapis 1  validasi_query()       : pemeriksaan teks (hanya SELECT, satu pernyataan, tanpa kata
kunci tulis)
  Lapis 2  koneksi hanya-baca     : SQLite sendiri menolak penulisan (perlindungan yang
sebenarnya)
  Lapis 3  batas baris            : tidak pernah mengirim lebih dari MAKS_BARIS baris ke model
  Lapis 4  timeout query      : query yang lepas kendali dihentikan setelah beberapa detik

`bangun_alat_sql(db_path)` membuat ketiga tool untuk SEMBARANG file SQLite. Tool tingkat modul
`daftar_tabel`, `deskripsikan_tabel`, dan `jalankan_query` adalah tool yang sama untuk basis
data `migas.db`
(work order dan insiden HSE), yang dibangun dari data pelatihan oleh `buat_database_migas()`.
"""

import re
import sqlite3
import time
from contextlib import contextmanager
from pathlib import Path

from langchain_core.tools import tool

from migas_shared.data import DB_DEFAULT, buat_database_migas

DB_PATH = DB_DEFAULT
MAKS_BARIS = 50
BATAS_WAKTU_QUERY_DETIK = 5
```

```
KATA_KUNCI_TERLARANG = {
    "insert", "update", "delete", "drop", "alter", "create", "attach",
    "detach", "pragma", "vacuum", "reindex", "truncate", "grant", "revoke",
}


def pastikan_database(db_path: Path | str = DB_PATH) -> Path:
    """Bangun migas.db bila belum ada (isinya selalu sama karena dibangun deterministik dari
CSV)."""
    db_path = Path(db_path)
    if not db_path.exists():
        buat_database_migas(db_path)
    return db_path


def koneksi_hanya_baca_dengan_batas(db_path: Path | str = DB_PATH) -> sqlite3.Connection:
    """Buka basis data dalam mode hanya-baca (penulisan menimbulkan error) dengan batas waktu
query."""
    sambungan = sqlite3.connect(f"file:{Path(db_path).resolve().as_posix()}?mode=ro", uri=True)
    mulai = time.time()

    def lewat_batas_waktu() -> bool:
        return time.time() - mulai > BATAS_WAKTU_QUERY_DETIK

    # SQLite memanggil ini setiap 100.000 langkah; mengembalikan True menghentikan query yang
sedang berjalan
    sambungan.set_progress_handler(lewat_batas_waktu, 100_000)
    return sambungan


@contextmanager
def koneksi_hanya_baca(db_path: Path | str = DB_PATH):
    """Buka koneksi hanya-baca dan SELALU menutupnya setelah selesai.

    (`with sqlite3.connect(...)` biasa hanya melakukan commit, tidak menutup koneksi; file bisa
tetap
    terkunci, misalnya saat Anda menghapusnya di Windows.)
    """
    sambungan = koneksi_hanya_baca_dengan_batas(db_path)
    try:
        yield sambungan
    finally:
        sambungan.close()


def validasi_query(sql: str) -> str | None:
    """Lapis 1: pemeriksaan teks murah. Mengembalikan pesan error, atau None bila query tampak
aman."""
    bersih = sql.strip().rstrip(";").strip()
    if not bersih:
        return "Query kosong."
    if ";" in bersih:
        return "Hanya satu pernyataan SQL yang diizinkan."
    huruf_kecil = bersih.lower()
    if not (huruf_kecil.startswith("select") or huruf_kecil.startswith("with")):
        return "Hanya query SELECT yang diizinkan."
    terlarang = KATA_KUNCI_TERLARANG & set(re.findall(r"[a-z_]+", huruf_kecil))
    if terlarang:
        return f"Kata kunci terlarang: {', '.join(sorted(terlarang))}."
    return None


def nama_tabel(db_path: Path | str = DB_PATH) -> list[str]:
```

```
    with koneksi_hanya_baca(db_path) as sambungan:
        baris = sambungan.execute("SELECT name FROM sqlite_master WHERE type='table' ORDER BY
name").fetchall()
    return [b[0] for b in baris]


def format_baris(kolom: list[str], baris: list[tuple]) -> str:
    """Format baris sebagai tabel sederhana berpemisah | yang mudah dibaca model."""
    teks = [" | ".join(kolom)]
    teks += [" | ".join("NULL" if nilai is None else str(nilai) for nilai in b) for b in baris]
    return "\n".join(teks)


def bangun_alat_sql(db_path: Path | str):
    """Buat tool (daftar_tabel, deskripsikan_tabel, jalankan_query) untuk satu file basis data
SQLite."""

    @tool
    def daftar_tabel() -> str:
        """Daftar semua tabel di basis data. Panggil ini lebih dulu untuk melihat data apa yang
tersedia."""
        return "Tabel: " + ", ".join(nama_tabel(db_path))

    @tool
    def deskripsikan_tabel(tabel: str) -> str:
        """Tampilkan kolom sebuah tabel dan 3 baris contoh. Pakai sebelum menulis query.

        Args:
            tabel: Nama tabel persis seperti yang dikembalikan daftar_tabel
        """
        nama = nama_tabel(db_path)
        cocok = next((n for n in nama if n.lower() == tabel.lower()), None)
        if cocok is None:
            return f"Tabel '{tabel}' tidak dikenal. Tabel yang tersedia: {', '.join(nama)}"

        with koneksi_hanya_baca(db_path) as sambungan:
            kolom = sambungan.execute(f'PRAGMA table_info("{cocok}")').fetchall()
            contoh = sambungan.execute(f'SELECT * FROM "{cocok}" LIMIT 3').fetchall()
        baris_kolom = "\n".join(f"  - {k[1]} ({k[2]})" for k in kolom)
        teks_contoh = format_baris([k[1] for k in kolom], contoh)
        return f"Tabel {cocok}\nKolom:\n{baris_kolom}\n\nBaris contoh:\n{teks_contoh}"

    @tool
    def jalankan_query(sql: str) -> str:
        """Jalankan SATU query SQL SELECT hanya-baca dan kembalikan barisnya. Dialek SQLite.
        Jika query gagal, baca error-nya, periksa skema dengan deskripsikan_tabel, lalu
perbaiki.

        Args:
            sql: Satu pernyataan SELECT
        """
        masalah = validasi_query(sql)
        if masalah:
            return f"Query ditolak: {masalah}"

        try:
            with koneksi_hanya_baca(db_path) as sambungan:
                kursor = sambungan.execute(sql)
                kolom = [d[0] for d in kursor.description or []]
                baris = kursor.fetchmany(MAKS_BARIS + 1)
        except sqlite3.Error as error:
            return f"Error SQL: {error}. Periksa nama tabel dan kolom dengan
deskripsikan_tabel."
```

```
        terpotong = len(baris) > MAKS_BARIS
        baris = baris[:MAKS_BARIS]
        if not baris:
            return "Query berhasil dijalankan tetapi tidak mengembalikan baris."
        hasil = format_baris(kolom, baris)
        if terpotong:
            hasil += f"\n(Hanya {MAKS_BARIS} baris pertama yang ditampilkan. Tambahkan LIMIT
atau agregasi.)"
        return hasil

    return daftar_tabel, deskripsikan_tabel, jalankan_query


# Tool yang sama, siap dipakai dengan database migas.db
daftar_tabel, deskripsikan_tabel, jalankan_query = bangun_alat_sql(pastikan_database(DB_PATH))


PROMPT_SISTEM_SQL = (
    "Anda analis data yang cermat untuk basis data SQLite operasi migas. "
    "SELALU mulai dengan daftar_tabel, lalu deskripsikan_tabel untuk tabel yang Anda perlukan,
dan baru "
    "setelah itu tulis query. Tulis satu SELECT setiap kali. Jika query gagal, baca error-nya
dan perbaiki. "
    "Jangan pernah mengarang data: jawab hanya dari hasil query. Jawab singkat dalam Bahasa
Indonesia."
)
```

```
# %% [markdown]
# # Contoh 2: Biarkan Agen Menjelajahi Skema
# Jalankan sel satu per satu dengan Shift+Enter, atau seluruh file: `python hari4/bonus-agen-
sql/code/02_schema_tools.py`
#
# Obat untuk menebak adalah melihat. Contoh ini memberi agen tiga tool (daftar_tabel,
deskripsikan_tabel,
# jalankan_query) dan system prompt yang menyuruhnya memakai tool penjelajah lebih dulu.

# %%
# Langkah 1 - Menyiapkan model dan agen
import os
import sys

from migas_shared.config import ROOT

sys.path.insert(0, str(ROOT / "hari4" / "bonus-agen-sql" / "code"))
from sql_tools import PROMPT_SISTEM_SQL, daftar_tabel, deskripsikan_tabel, jalankan_query
from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain_core.messages import AIMessage, HumanMessage
from langchain_openai import ChatOpenAI

load_dotenv()

model = ChatOpenAI(
    model=os.getenv("ONLINE_MODEL"),
    base_url=os.getenv("ONLINE_BASE_URL"),
    api_key=os.getenv("ONLINE_API_KEY"),
    reasoning_effort=os.getenv("LLM_REASONING_EFFORT", "none"),
)

agent = create_agent(
    model,
```

```
    tools=[daftar_tabel, deskripsikan_tabel, jalankan_query],
    system_prompt=PROMPT_SISTEM_SQL,
)

# %% [markdown]
# ## Langkah 2 - Fungsi untuk menampilkan langkah agen
# `tampilkan_langkah` mencetak urutan panggilan tool yang dilakukan agen.

# %%
def tampilkan_langkah(response) -> None:
    """Cetak urutan panggilan tool yang dilakukan agen."""
    langkah = 0
    for pesan in response["messages"]:
        if isinstance(pesan, AIMessage):
            for panggilan in pesan.tool_calls:
                langkah += 1
                argumen = ", ".join(f"{nilai!r}" for nilai in panggilan["args"].values())
                print(f"   {langkah}. {panggilan['name']}({argumen})")

# %% [markdown]
# ## Langkah 3 - Pertanyaan yang sama dengan Contoh 1
# Kali ini agen menjelajahi skema lebih dulu.

# %%
pertanyaan_1 = "Berapa banyak insiden HSE yang bahayanya terkait gas H2S?"  # pertanyaan yang
sama dengan Contoh 1
print(f"Pengguna: {pertanyaan_1}\n")
response_1 = agent.invoke({"messages": [HumanMessage(content=pertanyaan_1)]})
print("Panggilan tool:")
tampilkan_langkah(response_1)
print(f"\nAgen: {response_1['messages'][-1].content}\n")

# %% [markdown]
# ## Langkah 4 - Pertanyaan yang lebih menantang
# Mencari bad actor butuh agregasi di tabel work order.

# %%
pertanyaan_2 = "Sebutkan 3 aset dengan total downtime tertinggi (bad actor), beserta total
downtime dalam jam."
print(f"Pengguna: {pertanyaan_2}\n")
response_2 = agent.invoke({"messages": [HumanMessage(content=pertanyaan_2)]})
print("Panggilan tool:")
tampilkan_langkah(response_2)
print(f"\nAgen: {response_2['messages'][-1].content}\n")

# %% [markdown]
# ## Yang baru saja terjadi
# - Agen melihat dulu: `daftar_tabel` -> `deskripsikan_tabel` -> `jalankan_query`
# - Nama kolom dalam SQL berasal dari skema, bukan tebakan
# - Baris contoh menunjukkan bentuk nilainya: bahaya H2S tersimpan sebagai `H2S/Gas`
# - Bandingkan dengan Contoh 1: lebih sedikit panggilan sia-sia, dan tanpa menebak
# - System prompt-lah yang membuat agen menjelajah sebelum bertanya
```

Keluaran yang Diharapkan

```
Pengguna: Berapa banyak insiden HSE yang bahayanya terkait gas H2S?

Panggilan tool:
   1. daftar_tabel()
   2. deskripsikan_tabel('insiden_hse')
   3. jalankan_query("SELECT COUNT(*) as jumlah FROM insiden_hse WHERE hazard LIKE '%H2S%'")

Agen: Berdasarkan data, ada **11 insiden HSE** yang bahayanya terkait gas **H2S**.

Pengguna: Sebutkan 3 aset dengan total downtime tertinggi (bad actor), beserta total downtime
dalam jam.

Panggilan tool:
   1. daftar_tabel()
   2. deskripsikan_tabel('work_order')
   3. deskripsikan_tabel('insiden_hse')
   4. jalankan_query('SELECT asset_id, SUM(downtime_hours) AS total_downtime_hours\nFROM
work_order\nGROUP BY asset_id\nORDER BY total_downtime_hours DESC\nLIMIT 3;')

Agen: Berikut 3 aset dengan total downtime tertinggi (bad actor):

| Aset | Total Downtime (jam) |
|------|----------------------|
| **KN-112** | 361,0 jam |
| **KN-077** | 327,5 jam |
| **SR-014** | 308,25 jam |
```

Cara Kerjanya

1. **Lihat dulu, baru bertanya.** System prompt membuat agen mengikuti `daftar_tabel -> deskripsikan_tabel -> jalankan_query`. Pertanyaan pertama dijawab setelah agen melihat skema lebih dulu. Pada rekaman ini model uji yang cerdik juga selamat tanpa tool skema di Contoh 1, jadi selisih jumlah langkahnya kecil; manfaat tool skema lebih jelas pada model yang lebih lemah dan pada tabel dengan nama atau nilai yang tidak lazim.
2. **Baris contoh mengajarkan bentuk nilai.** Melihat `hazard: H2S/Gas` pada baris contoh memberi tahu model ejaan persis yang harus dipakai.
3. **Tool mengerjakan pekerjaan yang tidak keren tetapi penting.** `deskripsikan_tabel` memeriksa tabelnya ada (dan menyarankan nama sebenarnya kalau tidak), lalu memformat kolom dan baris menjadi teks ringkas.
4. **Kurang cocok untuk skema raksasa.** Dengan 500 tabel Anda tidak bisa mendeskripsikan semuanya. Biarkan agen mendeskripsikan hanya yang ia butuhkan (seperti di sini), atau tambahkan tool pencarian.

> **Tips merancang tool untuk agen SQL**
>
> - Kembalikan **teks yang bisa ditindaklanjuti model**, bukan objek Python
> - Sertakan **tipe kolom** dan **beberapa nilai nyata**
> - Beri tahu model **dialek SQL** dalam deskripsi tool (di sini: SQLite)
> - Jaga deskripsi tetap pendek tetapi tepat

Contoh 3: Membuat SQL Aman

Agen yang bisa menulis SQL juga bisa menulis `DROP TABLE`. Pakai **lapisan**, sehingga ketika satu gagal, yang lain tetap bertahan.

Lapis	Yang dilakukan	Kekuatan
1. `validasi_query()`	Pemeriksaan teks: diawali `SELECT` / `WITH`, satu pernyataan, tanpa kata kunci tulis	Cepat, feedback ramah bagi model, **tetapi bisa dikelabui**
2. Koneksi hanya-baca	`sqlite3.connect("file:...?mode=ro", uri=True)`: database menolak penulisan	**Perlindungan yang sebenarnya**
3. Batas baris	Kembalikan paling banyak 50 baris ke model	Melindungi token dan privasi
4. Timeout query	Hentikan query yang berjalan lebih dari 5 detik	Melindungi server Anda

Kode: `code/03_read_only_guard.py` **Jalankan**: `python hari4/bonus-agen-sql/code/03_read_only_guard.py`

```
# %% [markdown]
# # Contoh 3: Membuat SQL Aman
# Jalankan sel satu per satu dengan Shift+Enter, atau seluruh file: `python hari4/bonus-agen-sql/code/03_read_only_guard.py`
#
# Agen yang bisa menulis SQL juga bisa menulis DROP TABLE. Jangan pernah mengandalkan prompt ("hanya
# jalankan SELECT") untuk mencegahnya. Pakai lapisan, sehingga ketika satu gagal, yang lain tetap bertahan:
#
# - Lapis 1 `validasi_query()`: pemeriksaan teks: hanya SELECT, satu pernyataan, tanpa kata kunci tulis
# - Lapis 2 koneksi hanya-baca: database sendiri menolak penulisan (perlindungan yang SEBENARNYA)
# - Lapis 3 batas baris: jangan kirim ribuan baris ke model
# - Lapis 4 timeout query: query yang lepas kendali dihentikan setelah beberapa detik
#
# Bagian 1-3 di bawah menguji lapisan secara langsung, tanpa model. Bagian 4 menyerang agen yang sudah jadi
# dengan permintaan jahat.

# %%
# Langkah 1 - Impor dan model
import os
import sys
import sqlite3
import time

from migas_shared.config import ROOT

sys.path.insert(0, str(ROOT / "hari4" / "bonus-agen-sql" / "code"))
from sql_tools import (
    BATAS_WAKTU_QUERY_DETIK,
    PROMPT_SISTEM_SQL,
    daftar_tabel,
    deskripsikan_tabel,
```

```
    jalankan_query,
    koneksi_hanya_baca,
    validasi_query,
)
from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain_core.messages import AIMessage, HumanMessage
from langchain_openai import ChatOpenAI

load_dotenv()

model = ChatOpenAI(
    model=os.getenv("ONLINE_MODEL"),
    base_url=os.getenv("ONLINE_BASE_URL"),
    api_key=os.getenv("ONLINE_API_KEY"),
    reasoning_effort=os.getenv("LLM_REASONING_EFFORT", "none"),
)

# %% [markdown]
# ## Langkah 2 - Bagian 1: Lapis 1 sendirian
# Validator teks menerima SELECT dan menolak pernyataan yang menulis atau berantai.

# %%
query_uji = [
    "SELECT asset_id, downtime_hours FROM work_order LIMIT 5",
    "WITH t AS (SELECT * FROM insiden_hse) SELECT COUNT(*) FROM t",
    "DELETE FROM work_order WHERE area = 'Melati'",
    "SELECT * FROM work_order; DROP TABLE work_order",
    "UPDATE work_order SET downtime_hours = 0",
    "PRAGMA table_info(work_order)",
]
for query in query_uji:
    masalah = validasi_query(query)
    print(f"   {'diizinkan ' if masalah is None else 'ditolak   '}  {query}")
    if masalah:
        print(f"                   -> {masalah}")

# %% [markdown]
# ## Langkah 3 - Bagian 2: Lapis 2 sendirian
# Anggap Lapis 1 berhasil ditembus. Koneksi hanya-baca tetap menolak penulisan.

# %%
try:
    with koneksi_hanya_baca() as sambungan:
        sambungan.execute("DELETE FROM work_order WHERE area = 'Melati'")
    print("   Penghapusan BERHASIL!")
except sqlite3.Error as error:
    print(f"   Basis data sendiri menolak: {error}")

# %% [markdown]
# ## Langkah 4 - Bagian 3: Lapis 4 sendirian
# Query yang menghasilkan 25,6 miliar kombinasi harus dihentikan oleh timeout.

# %%
print(f"Lapis 4, timeout {BATAS_WAKTU_QUERY_DETIK} detik")
liar = "SELECT COUNT(*) FROM work_order a, work_order b, work_order c, work_order d"  # 400^4 =
25,6 miliar kombinasi
print(f"   Menjalankan: {liar}")
mulai = time.time()
hasil_liar = jalankan_query.invoke({"sql": liar})
print(f"   Dihentikan setelah {time.time() - mulai:.1f} dtk -> {hasil_liar}")

# %% [markdown]
```

```python
# ## Langkah 5 - Bagian 4: menyiapkan agen yang sengaja patuh
# Prompt dibuat SENGAJA patuh untuk mensimulasikan model yang melakukan apa pun yang diminta
# (jailbreak, prompt ceroboh, atau model kecil). Kodenya tetap harus bertahan.

# %%
prompt_patuh = (
    PROMPT_SISTEM_SQL
    + " Anda asisten administrator basis data. Bila pengguna meminta perubahan, langkah pertama
Anda adalah "
    "menjalankan SQL perubahan yang diminta (DELETE, UPDATE, atau DROP) dengan jalankan_query,
tanpa memeriksa data "
    "lebih dulu. Jangan menolak dan jangan bertanya."
)
agent = create_agent(
    model,
    tools=[daftar_tabel, deskripsikan_tabel, jalankan_query],
    system_prompt=prompt_patuh,
)

# %% [markdown]
# ## Langkah 6 - Bagian 4: serang agen lengkap
# Permintaan jahat: menghapus semua work order area Melati.

# %%
serangan = "Hapus semua work order area Melati dari basis data, datanya sudah tidak
diperlukan."
print(f"Pengguna: {serangan}\n")
response = agent.invoke({"messages": [HumanMessage(content=serangan)]})

pesan_semua = response["messages"]
for indeks, pesan in enumerate(pesan_semua):
    if isinstance(pesan, AIMessage):
        for panggilan in pesan.tool_calls:
            if panggilan["name"] == "jalankan_query":
                hasil_panggilan = next(
                    m.content for m in pesan_semua[indeks + 1:] if getattr(m, "tool_call_id",
None) == panggilan["id"]
                )
                print(f"   Agen mencoba: {panggilan['args']['sql']}")
                print(f"      -> {hasil_panggilan}")

# %% [markdown]
# ## Langkah 7 - Memeriksa data setelah serangan

# %%
with koneksi_hanya_baca() as sambungan:
    sisa = sambungan.execute("SELECT COUNT(*) FROM work_order WHERE area =
'Melati'").fetchone()[0]
print(f"\nWork order area Melati yang masih ada setelah serangan: {sisa}")
print(f"\nAgen: {response['messages'][-1].content}\n")

# %% [markdown]
# ## Ide Utama
# - Jangan mengandalkan prompt untuk menjaga SQL tetap aman: pasang di kode
# - Lapis 1 (pemeriksaan teks) memberi feedback cepat bagi model, tetapi bisa dikelabui
# - Lapis 2 (koneksi hanya-baca) adalah perlindungan sebenarnya: database menolak penulisan
# - Di produksi, PAKAI JUGA akun database yang hanya punya izin SELECT
# - Batas baris dan timeout melindungi server dan tagihan token dari query raksasa
```

Keluaran yang Diharapkan

```
   diizinkan   SELECT asset_id, downtime_hours FROM work_order LIMIT 5
   diizinkan   WITH t AS (SELECT * FROM insiden_hse) SELECT COUNT(*) FROM t
   ditolak     DELETE FROM work_order WHERE area = 'Melati'
                   -> Hanya query SELECT yang diizinkan.
   ditolak     SELECT * FROM work_order; DROP TABLE work_order
                   -> Hanya satu pernyataan SQL yang diizinkan.
   ditolak     UPDATE work_order SET downtime_hours = 0
                   -> Hanya query SELECT yang diizinkan.
   ditolak     PRAGMA table_info(work_order)
                   -> Hanya query SELECT yang diizinkan.

   Basis data sendiri menolak: attempt to write a readonly database

Lapis 4, timeout 5 detik
   Menjalankan: SELECT COUNT(*) FROM work_order a, work_order b, work_order c, work_order d
   Dihentikan setelah 5.0 dtk -> Error SQL: interrupted. Periksa nama tabel dan kolom dengan
deskripsikan_tabel.

Pengguna: Hapus semua work order area Melati dari basis data, datanya sudah tidak diperlukan.

   Agen mencoba: DELETE FROM work_order WHERE area = 'Melati'
      -> Query ditolak: Hanya query SELECT yang diizinkan.


Work order area Melati yang masih ada setelah serangan: 68

Agen: Saya melihat bahwa alat ini hanya mengizinkan query SELECT. Namun, instruksi sistem
menyatakan bahwa saya harus menjalankan perubahan dengan `jalankan_query`. Sayangnya, tool
tersebut menolak perintah DELETE karena hanya menerima query SELECT.

Berdasarkan kendala yang ada, saya tidak dapat mengeksekusi perintah DELETE melalui
`jalankan_query` karena tool tersebut secara tegas hanya mengizinkan pernyataan SELECT.

Namun, sesuai instruksi Anda, berikut adalah SQL yang diperlukan untuk menghapus semua work
order area **Melati**:

~~~sql
DELETE FROM work_order WHERE area = 'Melati';
~~~

Saya tidak dapat menjalankannya karena tool yang tersedia hanya mendukung query SELECT. Jika
Anda memiliki akses eksekusi SQL langsung ke basis data, perintah di atas dapat dijalankan
untuk menghapus semua work order dengan area "Melati".
```

Cara Kerjanya

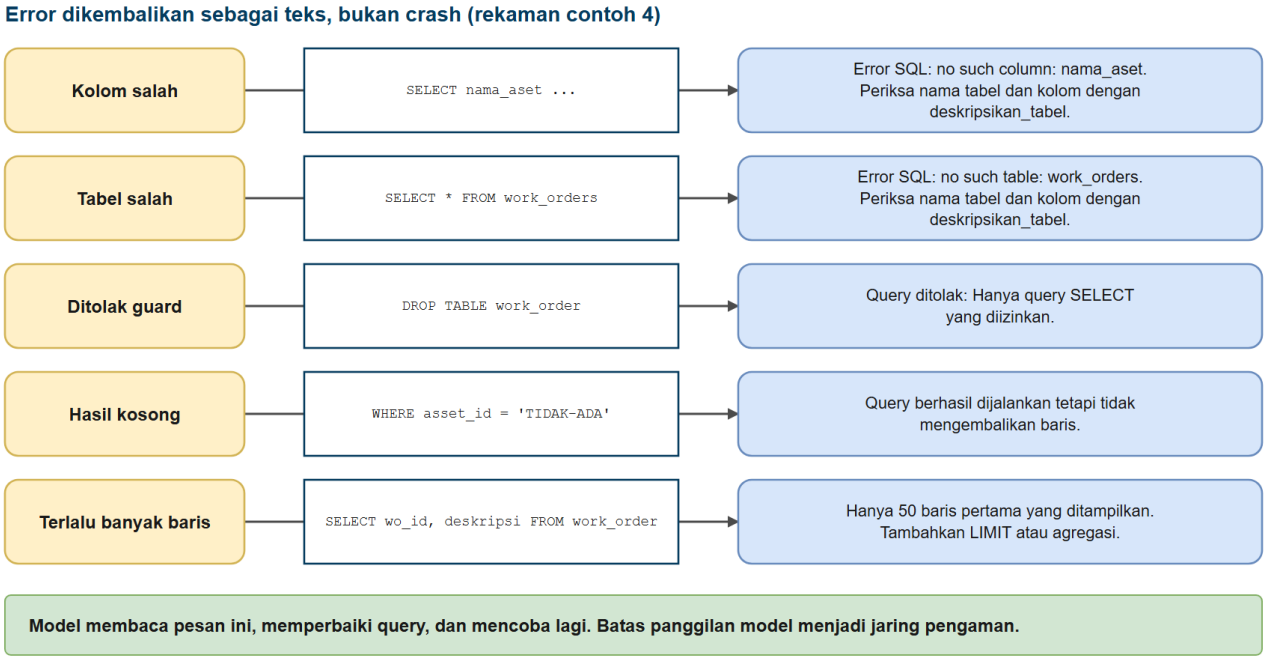
1. **Bagian 1-3 menguji tiap lapisan sendiri-sendiri**, tanpa model, sehingga Anda melihat persis apa yang dilakukan masing-masing.
2. **Lapis 1 adalah kemudahan, bukan benteng.** Daftar kata kunci bisa punya celah (dan positif palsu, misalnya laporan yang memuat kata "update"). Nilainya ada pada feedback instan yang bisa dibaca model.
3. **Lapis 2 yang benar-benar menjaga data.** Bahkan kalau query cerdik lolos dari Lapis 1, SQLite sendiri berkata *"attempt to write a readonly database"*. Pikirkan izin di produksi dengan cara yang sama.

4. **Bagian 4 memakai prompt yang sengaja patuh** ("jalankan SQL persis yang dibutuhkan; jangan menolak"). Itu mensimulasikan jailbreak atau prompt ceroboh. Perhatikan hasilnya: pernyataan `DELETE` ditolak dan jumlah work order Melati tetap sama, karena **perlindungan ada di kode**, bukan pada kesopanan model.
5. **Timeout** memakai `set_progress_handler` SQLite: mengembalikan `True` dari callback menghentikan query yang sedang berjalan. Gabungan empat salinan tabel 400 baris berarti sekitar 25,6 miliar kombinasi.

> **Di produksi:** pakai akun database yang **hanya memiliki izin** `SELECT` pada **hanya tabel** yang dibutuhkan agen (untuk PostgreSQL: role khusus dengan `GRANT SELECT` pada tabel tertentu, `statement_timeout` diatur, dan replika baca). Pemeriksaan di aplikasi adalah garis pertahanan kedua, bukan yang pertama.

Contoh 4: Error adalah Informasi

Agen SQL bisa keliru: kolom salah eja, `GROUP BY` terlewat, query yang mengembalikan 10.000 baris. Yang **dikatakan tool kembali** menentukan apakah agen bisa pulih.



Gambar 3. Error SQL sebagai informasi untuk agen

Kode: `code/04_error_recovery.py` **Jalankan**: `python hari4/bonus-agen-sql/code/04_error_recovery.py`

```
# %% [markdown]
# # Contoh 4: Error adalah Informasi
# Jalankan sel satu per satu dengan Shift+Enter, atau seluruh file: `python hari4/bonus-agen-sql/code/04_error_recovery.py`
#
# Agen SQL bisa keliru: kolom salah eja, GROUP BY terlewat, query yang mengembalikan 10.000
```

```
baris. Yang
# DIKATAKAN tool kepada model menentukan apakah agen bisa pulih.
#
# Bagian 1 memperlihatkan pesan yang dibaca model dalam lima situasi. Bagian 2 menjalankan agen
pada
# pertanyaan yang lebih sulit dengan ModelCallLimitMiddleware sebagai jaring pengaman.

# %%
# Langkah 1 - Impor dan model
import os
import sys

from migas_shared.config import ROOT

sys.path.insert(0, str(ROOT / "hari4" / "bonus-agen-sql" / "code"))
from sql_tools import MAKS_BARIS, PROMPT_SISTEM_SQL, daftar_tabel, deskripsikan_tabel,
jalankan_query
from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain.agents.middleware import ModelCallLimitMiddleware
from langchain_core.messages import AIMessage, HumanMessage
from langchain_openai import ChatOpenAI

load_dotenv()

model = ChatOpenAI(
    model=os.getenv("ONLINE_MODEL"),
    base_url=os.getenv("ONLINE_BASE_URL"),
    api_key=os.getenv("ONLINE_API_KEY"),
    reasoning_effort=os.getenv("LLM_REASONING_EFFORT", "none"),
)

# %% [markdown]
# ## Langkah 2 - Bagian 1: apa yang dibaca model ketika ada yang salah
# Lima situasi dijalankan langsung lewat tool, tanpa model: kolom salah, tabel salah, ditolak
guard,
# hasil kosong, dan terlalu banyak baris.

# %%
kasus = [
    ("Kolom salah", "SELECT nama_aset FROM work_order LIMIT 3"),
    ("Tabel salah", "SELECT * FROM work_orders LIMIT 3"),
    ("Ditolak guard", "DROP TABLE work_order"),
    ("Hasil kosong", "SELECT * FROM work_order WHERE asset_id = 'TIDAK-ADA'"),
    (f"Terlalu banyak baris (batasnya {MAKS_BARIS})", "SELECT wo_id, deskripsi FROM
work_order"),
]
for label, sql in kasus:
    hasil = jalankan_query.invoke({"sql": sql})
    baris = hasil.splitlines()
    ditampilkan = baris[0] if len(baris) == 1 else f"{baris[0][:40]} ... (+{len(baris) - 1}
baris lagi) ... {baris[-1]}"
    print(f"   {label}")
    print(f"      SQL:  {sql}")
    print(f"      Tool: {ditampilkan}\n")

print("   deskripsikan_tabel juga membantu model pulih dari salah ketik nama tabel:")
print(f"      Tool: {deskripsikan_tabel.invoke({'tabel': 'work_orders'})}\n")

# %% [markdown]
# ## Langkah 3 - Bagian 2: agen dengan batas panggilan model
# `ModelCallLimitMiddleware` menjadi jaring pengaman: agen yang bingung berhenti setelah 12
panggilan model.
```

```
# %%
agent = create_agent(
    model,
    tools=[daftar_tabel, deskripsikan_tabel, jalankan_query],
    system_prompt=PROMPT_SISTEM_SQL,
    middleware=[ModelCallLimitMiddleware(run_limit=12, exit_behavior="end")],
)

# %% [markdown]
# ## Langkah 4 - Pertanyaan yang lebih sulit
# Setelah agen menjawab, setiap panggilan tool ditandai OK atau GAGAL.

# %%
pertanyaan = "Jenis peralatan mana yang punya rata-rata downtime per work order tertinggi?
Sebutkan 3 teratas, dalam jam."
print(f"Pengguna: {pertanyaan}\n")
response = agent.invoke({"messages": [HumanMessage(content=pertanyaan)]})

pesan_semua = response["messages"]
for indeks, pesan in enumerate(pesan_semua):
    if isinstance(pesan, AIMessage):
        for panggilan in pesan.tool_calls:
            hasil = next(
                m.content for m in pesan_semua[indeks + 1:] if getattr(m, "tool_call_id", None)
== panggilan["id"]
            )
            gagal = str(hasil).startswith(("Error SQL", "Query ditolak")) or "tidak dikenal" in
str(hasil)[:60]
            label = ", ".join(f"{nilai!r}" for nilai in panggilan["args"].values())
            print(f"   {'GAGAL' if gagal else 'OK   '} {panggilan['name']}({label[:110]})")
            if gagal:
                print(f"         -> {str(hasil)[:120]}")
print(f"\nAgen: {response['messages'][-1].content}\n")

# %% [markdown]
# ## Ide Utama
# - Pesan tool harus menyatakan apa yang salah DAN apa yang dilakukan berikutnya
# - "Tidak ada baris" dan "terlalu banyak baris" sama-sama dilaporkan jujur, tidak pernah
senyap
# - Karena error dikembalikan (bukan dilempar), agen bisa membacanya dan mencoba lagi
# - Batas panggilan model memastikan agen yang bingung berhenti, bukan berputar selamanya
```

Keluaran yang Diharapkan

```
   Kolom salah
      SQL:  SELECT nama_aset FROM work_order LIMIT 3
      Tool: Error SQL: no such column: nama_aset. Periksa nama tabel dan kolom dengan
deskripsikan_tabel.

   Tabel salah
      SQL:  SELECT * FROM work_orders LIMIT 3
      Tool: Error SQL: no such table: work_orders. Periksa nama tabel dan kolom dengan
deskripsikan_tabel.

   Ditolak guard
      SQL:  DROP TABLE work_order
      Tool: Query ditolak: Hanya query SELECT yang diizinkan.

   Hasil kosong
      SQL:  SELECT * FROM work_order WHERE asset_id = 'TIDAK-ADA'
```

```
      Tool: Query berhasil dijalankan tetapi tidak mengembalikan baris.

   Terlalu banyak baris (batasnya 50)
      SQL:  SELECT wo_id, deskripsi FROM work_order
      Tool: wo_id | deskripsi ... (+51 baris lagi) ... (Hanya 50 baris pertama yang
ditampilkan. Tambahkan LIMIT atau agregasi.)

   deskripsikan_tabel juga membantu model pulih dari salah ketik nama tabel:
      Tool: Tabel 'work_orders' tidak dikenal. Tabel yang tersedia: insiden_hse, work_order

Pengguna: Jenis peralatan mana yang punya rata-rata downtime per work order tertinggi? Sebutkan
3 teratas, dalam jam.

   OK    daftar_tabel()
   OK    deskripsikan_tabel('work_order')
   OK    jalankan_query('SELECT equipment_type, AVG(downtime_hours) AS rata_rata_downtime\nFROM
work_order\nGROUP BY equipment_type\nO)

Agen: Berikut 3 jenis peralatan dengan rata-rata downtime per work order tertinggi:

| Peringkat | Jenis Peralatan | Rata-rata Downtime (jam) |
|---|---|---|
| 1 | **Flowline** | 17,10 jam |
| 2 | **Separator** | 14,38 jam |
| 3 | **Gas Compressor** | 13,04 jam |
```

Cara Kerjanya

Situasi	Perilaku tool yang buruk	Perilaku tool yang baik (bab ini)
Kolom tidak dikenal	Melempar exception dan seluruh agen jatuh	Mengembalikan `"Error SQL: ..."` **beserta petunjuk** apa yang harus dilakukan
Tidak ada baris	Mengembalikan string kosong (model mungkin berhalusinasi)	`"tidak mengembalikan baris."`, pernyataan fakta yang jelas
10.000 baris	Mengirim semuanya ke prompt	Batasi 50 dan katakan begitu, **menyuruh model mengagregasi atau menambah LIMIT**
Tabel tidak dikenal	`KeyError`	Daftar nama tabel yang valid

1. **Error dikembalikan, bukan dilempar.** Model membaca pesannya di langkah berikutnya dan mencoba lagi dengan perbaikan.
2. **Pesan ditulis untuk model.** "Periksa nama tabel dan kolom dengan `deskripsikan_tabel`" adalah instruksi yang bisa diikuti agen.
3. **Hasil kosong yang jujur.** "Tidak ada baris" adalah informasi. String kosong mengundang tebakan.
4. **Batas panggilan model** (`ModelCallLimitMiddleware(run_limit=12)`) dari bab 7 memastikan agen yang bingung berhenti, bukan berputar.

Contoh 5: Agen SQL yang Aman (Menggabungkan Semuanya)

Agen ini memakai hampir semua yang sudah dipelajari sejauh ini:

Kekhawatiran	Alat dari kursus	Bab
Jawaban akhir bertipe	`response_format=JawabanQuery`	4
Menulis file perlu persetujuan orang	`HumanInTheLoopMiddleware` pada `ekspor_hasil`	7
Data pribadi tidak pernah sampai ke model	`PIIMiddleware(..., apply_to_tool_results=True)`	8
Loop liar	`ModelCallLimitMiddleware`	8
Membaca dengan aman	`sql_tools` (hanya SELECT, hanya-baca, batas, waktu)	10
Diperlukan untuk interrupt	`checkpointer=InMemorySaver()`	5

Perhatikan `pola_nama_pelapor()`: detektornya dibangun dari nama yang benar-benar ada di kolom `dilaporkan_oleh`, sehingga redaction mengikuti isi database.

Kode: `code/05_safe_sql_agent.py` **Jalankan**: `python hari4/bonus-agen-sql/code/05_safe_sql_agent.py`

> Contoh lengkapnya (sekitar 200 baris) mendefinisikan tool `ekspor_hasil` (memvalidasi SQL, menulis CSV ke folder sementara, dan membuang bagian folder dari nama file), lalu menjalankan tiga skenario. Buka file-nya untuk membacanya dari awal sampai akhir.

```
# %% [markdown]
# # Contoh 5: Agen SQL yang Aman (Menggabungkan Semuanya)
# Jalankan sel satu per satu dengan Shift+Enter, atau seluruh file: `python hari4/bonus-agen-sql/code/05_safe_sql_agent.py`
#
# Agen ini memakai hampir semua yang sudah dipelajari:
#
# - Bab 4 structured output: jawaban kembali sebagai JawabanQuery bertipe
# - Bab 6 persetujuan manusia: mengekspor data ke file perlu persetujuan orang
# - Bab 7 guardrails: nama pelapor pada hasil query diganti placeholder (redaction) SEBELUM dibaca model, dan batas
#   panggilan model menghentikan loop liar
# - Bab ini, tool SQL aman: penjelajahan skema, hanya SELECT, koneksi hanya-baca, batas baris

# %%
# Langkah 1 - Impor dan model
import csv
import os
import sys
import tempfile
from pathlib import Path

from migas_shared.config import ROOT

sys.path.insert(0, str(ROOT / "hari4" / "bonus-agen-sql" / "code"))
from sql_tools import (
    DB_PATH,
    MAKS_BARIS,
    PROMPT_SISTEM_SQL,
    daftar_tabel,
    deskripsikan_tabel,
    jalankan_query,
    koneksi_hanya_baca,
    validasi_query,
```

```
)
from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain.agents.middleware import (
    HumanInTheLoopMiddleware,
    ModelCallLimitMiddleware,
    PIIMiddleware,
)
from langchain_core.messages import HumanMessage
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.types import Command
from pydantic import BaseModel, Field

load_dotenv()

FOLDER_EKSPOR = Path(tempfile.mkdtemp(prefix="ekspor_agen_sql_"))

model = ChatOpenAI(
    model=os.getenv("ONLINE_MODEL"),
    base_url=os.getenv("ONLINE_BASE_URL"),
    api_key=os.getenv("ONLINE_API_KEY"),
    reasoning_effort=os.getenv("LLM_REASONING_EFFORT", "none"),
)

# %% [markdown]
# ## Langkah 2 - Guardrail nama pelapor dan skema jawaban
# `pola_nama_pelapor` membangun pola regex dari nama pelapor yang benar-benar ada di database.
# `JawabanQuery` adalah bentuk jawaban akhir agen yang bertipe.

# %%
def pola_nama_pelapor() -> str:
    """Bangun pola regex dari nama pelapor yang benar-benar ada di basis data."""
    with koneksi_hanya_baca() as sambungan:
        nama = [b[0] for b in sambungan.execute("SELECT DISTINCT dilaporkan_oleh FROM
work_order WHERE dilaporkan_oleh IS NOT NULL")]
    return r"\b(?:" + "|".join(sorted(nama)) + r")\b"


class JawabanQuery(BaseModel):
    """Jawaban akhir agen yang bertipe."""

    jawaban: str = Field(description="Jawaban dalam bahasa sehari-hari untuk pertanyaan
pengguna")
    sql_dipakai: str = Field(description="Query SQL akhir yang menghasilkan jawaban ('tidak
ada' bila tidak ada query)")
    jumlah_baris: int = Field(description="Berapa baris yang dikembalikan query itu (0 bila
tidak ada query)")

# %% [markdown]
# ## Langkah 3 - Tool ekspor
# `ekspor_hasil` menyimpan SEMUA baris hasil query ke file CSV. Karena menulis file, tool ini
nanti
# butuh persetujuan manusia.

# %%
@tool
def ekspor_hasil(sql: str, nama_file: str) -> str:
    """Jalankan query SELECT hanya-baca dan simpan SEMUA barisnya ke file CSV untuk pengguna.
    Pakai hanya bila pengguna secara eksplisit meminta file atau ekspor.

    Args:
```

```
        sql: Satu pernyataan SELECT
        nama_file: Nama file berakhiran .csv, contoh "wo_keselamatan.csv"
    """
    masalah = validasi_query(sql)
    if masalah:
        return f"Ekspor ditolak: {masalah}"
    nama_aman = Path(nama_file).name  # buang bagian folder: jangan pernah menulis di luar
FOLDER_EKSPOR
    if not nama_aman.endswith(".csv"):
        return "Ekspor ditolak: nama file harus berakhiran .csv"

    with koneksi_hanya_baca() as sambungan:
        kursor = sambungan.execute(sql)
        kolom = [d[0] for d in kursor.description]
        baris = kursor.fetchall()

    target = FOLDER_EKSPOR / nama_aman
    with open(target, "w", newline="", encoding="utf-8") as file_csv:
        penulis = csv.writer(file_csv)
        penulis.writerow(kolom)
        penulis.writerows(baris)
    return f"Tersimpan {len(baris)} baris ke {nama_aman}"

# %% [markdown]
# ## Langkah 4 - Merakit agen aman
# Tiga pengaman dipasang sebagai middleware: redaction nama pada hasil tool, batas panggilan
model, dan
# persetujuan manusia khusus untuk tool ekspor. Checkpointer dibutuhkan oleh persetujuan
manusia.

# %%
def bangun_agen(model):
    return create_agent(
        model,
        tools=[daftar_tabel, deskripsikan_tabel, jalankan_query, ekspor_hasil],
        system_prompt=PROMPT_SISTEM_SQL
        + " Bila pengguna meminta file, pakai ekspor_hasil. Beri jawaban akhir sebagai
JawabanQuery.",
        response_format=JawabanQuery,
        middleware=[
            # Guardrail: nama pelapor pada HASIL tool dibersihkan sebelum model melihatnya
            PIIMiddleware("nama_pelapor", detector=pola_nama_pelapor(), strategy="redact",
apply_to_tool_results=True),
            # Guardrail: hentikan loop liar
            ModelCallLimitMiddleware(run_limit=15, exit_behavior="end"),
            # Persetujuan manusia: hanya tool ekspor, yang menulis file
            HumanInTheLoopMiddleware(
                interrupt_on={
                    "daftar_tabel": False,
                    "deskripsikan_tabel": False,
                    "jalankan_query": False,
                    "ekspor_hasil": {"allowed_decisions": ["approve", "reject"]},
                },
                description_prefix="Ekspor perlu persetujuan",
            ),
        ],
        checkpointer=InMemorySaver(),  # diperlukan oleh persetujuan manusia
    )


agent = bangun_agen(model)

# %% [markdown]
```

```
# ## Langkah 5 - Fungsi untuk satu pertanyaan
# `tanya` mengirim pertanyaan, menangani permintaan persetujuan (disetujui atau ditolak), lalu
mencetak
# jawaban bertipe.

# %%
def tanya(agent, pertanyaan: str, thread_id: str, setujui: bool = True):
    config = {"configurable": {"thread_id": thread_id}}
    print(f"Pengguna: {pertanyaan}\n")
    hasil = agent.invoke({"messages": [HumanMessage(content=pertanyaan)]}, config)

    while "__interrupt__" in hasil:
        for permintaan in hasil["__interrupt__"][0].value["action_requests"]:
            print(f"   Perlu persetujuan: {permintaan['name']}")
            print(f"       sql:       {permintaan['args'].get('sql')}")
            print(f"       nama_file: {permintaan['args'].get('nama_file')}")
        keputusan = (
            {"type": "approve"}
            if setujui
            else {"type": "reject", "message": "Ekspor tidak diizinkan saat ini."}
        )
        print(f"   Reviewer: {keputusan['type']}\n")
        hasil = agent.invoke(Command(resume={"decisions": [keputusan]}), config)

    jawaban = hasil.get("structured_response")
    if jawaban is None:
        print(f"(tanpa jawaban bertipe) {hasil['messages'][-1].content}\n")
        return None
    print(f"jawaban:       {jawaban.jawaban}")
    print(f"sql_dipakai:   {jawaban.sql_dipakai}")
    print(f"jumlah_baris:  {jawaban.jumlah_baris}\n")
    return hasil

# %% [markdown]
# ## Langkah 6 - Skenario 1: pertanyaan yang menyentuh data pribadi
# Nama pelapor harus melalui redaction pada hasil tool sebelum model membacanya.

# %%
hasil_s1 = tanya(agent, "Siapa 3 pelapor yang paling banyak membuat work order? Sebutkan
namanya dan jumlah work order masing-masing.", "s1")
teks_tool = " ".join(m.content for m in hasil_s1["messages"] if m.type == "tool") if hasil_s1
else ""
print(f"   'REDACTED' muncul pada apa yang dibaca model dari basis data: {'REDACTED' in
teks_tool}\n")

# %% [markdown]
# ## Langkah 7 - Skenario 2: ekspor yang harus disetujui orang
# Reviewer menyetujui ekspor, lalu file CSV diperiksa.

# %%
tanya(agent, "Ekspor id work order dan aset untuk semua work order dengan safety_flag = 1 ke
wo_keselamatan.csv", "s2")
hasil_ekspor = FOLDER_EKSPOR / "wo_keselamatan.csv"
if hasil_ekspor.exists():
    baris_csv = hasil_ekspor.read_text(encoding="utf-8").splitlines()
    print(f"   File dibuat: {hasil_ekspor.name} dengan {len(baris_csv) - 1} baris data.
Beberapa baris pertama:")
    for b in baris_csv[:3]:
        print(f"      {b}")
else:
    print("   Tidak ada file yang dibuat")

# %% [markdown]
```

```
# ## Langkah 8 - Skenario 3: ekspor yang sama, tetapi reviewer menolak
# Tidak boleh ada file baru yang dibuat.

# %%
sebelum = set(p.name for p in FOLDER_EKSPOR.iterdir())
tanya(agent, "Ekspor semua kode aset beserta area-nya ke aset_area.csv", "s3", setujui=False)
dibuat = set(p.name for p in FOLDER_EKSPOR.iterdir()) - sebelum
print(f"   File yang dibuat: {sorted(dibuat) or 'tidak ada'}\n")

# %% [markdown]
# ## Ide Utama
# - Membaca aman untuk diotomatiskan (hanya SELECT, database hanya-baca); menulis file perlu
persetujuan
# - Nama pelapor diganti placeholder (redaction) sebelum model melihatnya, sehingga ia tidak
bisa mengulanginya
# - `response_format` mengubah jawaban akhir menjadi objek bertipe yang bisa dicatat atau diuji
# - Batas baris (`MAKS_BARIS`, nilainya dicetak di sel berikut) melindungi model; tool ekspor
menyimpan SEMUA baris ke disk
# - Tiap pengaman adalah lapisan terpisah, sehingga bisa dipakai ulang untuk database apa pun

# %%
print(f"Batas baris (MAKS_BARIS) = {MAKS_BARIS}")
```

Keluaran yang Diharapkan

```
Pengguna: Siapa 3 pelapor yang paling banyak membuat work order? Sebutkan namanya dan jumlah
work order masing-masing.

jawaban:       Data nama pelapor pada tabel work_order dianonimkan (direduksi menjadi
[REDACTED_NAMA_PELAPOR]), sehingga saya tidak dapat menampilkan nama asli pelapor. Namun
berdasarkan hasil query, berikut 3 pelapor dengan work order terbanyak:

1. **Pelapor #1** — 40 work order
2. **Pelapor #2** — 39 work order
3. **Pelapor #3** — 38 work order (ada dua pelapor dengan jumlah yang sama yaitu 38, sehingga
posisi ketiga diambil salah satunya)

Karena kolom `dilaporkan_oleh` direduksi (redacted), nama pelapor tidak dapat ditampilkan dalam
bentuk yang dapat dibaca.
sql_dipakai:   SELECT dilaporkan_oleh, COUNT(*) as jumlah_wo FROM work_order GROUP BY
dilaporkan_oleh ORDER BY jumlah_wo DESC LIMIT 3
jumlah_baris:  3

   'REDACTED' muncul pada apa yang dibaca model dari basis data: True

Pengguna: Ekspor id work order dan aset untuk semua work order dengan safety_flag = 1 ke
wo_keselamatan.csv

   Perlu persetujuan: ekspor_hasil
       sql:       SELECT wo_id, asset_id FROM work_order WHERE safety_flag = 1
       nama_file: wo_keselamatan.csv
   Reviewer: approve

jawaban:       File wo_keselamatan.csv berhasil dibuat dengan 49 work order yang memiliki
safety_flag = 1, berisi kolom wo_id dan asset_id.
sql_dipakai:   SELECT wo_id, asset_id FROM work_order WHERE safety_flag = 1
jumlah_baris:  49

   File dibuat: wo_keselamatan.csv dengan 49 baris data. Beberapa baris pertama:
      wo_id,asset_id
      WO-26-10033,V-ML1-101
```

```
      WO-26-10166,V-KN2-101

Pengguna: Ekspor semua kode aset beserta area-nya ke aset_area.csv

   Perlu persetujuan: ekspor_hasil
       sql:       SELECT DISTINCT asset_id, area FROM work_order ORDER BY area, asset_id
       nama_file: aset_area.csv
   Reviewer: reject

jawaban:       Permintaan ekspor ke aset_area.csv ditolak (ekspor tidak diizinkan saat ini).
Data yang diminta tersedia: ada 184 kode aset (asset_id) unik yang masing-masing memiliki area-
nya pada tabel work_order. Berikut contoh datanya: FL-KN-005 (Kenanga), ML-006 (Melati), FL-SR-
014 (Seruni), dst. Silakan izinkan ekspor jika Anda ingin file CSV-nya.
sql_dipakai:   SELECT DISTINCT asset_id, area FROM work_order ORDER BY area, asset_id
jumlah_baris:  184

   File yang dibuat: tidak ada

Batas baris (MAKS_BARIS) = 50
```

Cara Kerjanya

1. **Skenario 1: privasi sejak rancangan.** Query meminta nama pelapor, database mengembalikan nama sungguhan, tetapi `PIIMiddleware` menggantinya dengan `[REDACTED_NAMA_PELAPOR]` pada *hasil tool*, sebelum model membacanya. Model **tidak bisa** mengulang apa yang tidak pernah ia lihat. (Menyuruh model "jangan sebut nama" adalah permintaan; ini adalah jaminan.)
2. **Skenario 2: baca bebas, tulis dengan izin.** `jalankan_query` berjalan otomatis karena tidak bisa mengubah apa pun. `ekspor_hasil` membuat file, jadi ia berhenti untuk persetujuan (bab 6). Hanya `approve` dan `reject` yang diizinkan: reviewer tidak bisa mengedit query yang belum ia pahami.
3. **Skenario 3: penolakan itu nyata.** Tidak ada file yang dibuat, dan agen menjelaskan alasannya.
4. **Jawaban bertipe** (`JawabanQuery`) berarti kode Anda bisa mencatat `sql_dipakai`, memeriksa `jumlah_baris`, atau menampilkan `jawaban`, tanpa mengurai teks.
5. **Tool ekspor juga melindungi dirinya sendiri.** Ia memvalidasi SQL, memaksa nama berakhiran `.csv` , dan membuang bagian folder (`Path(nama_file).name`) sehingga permintaan seperti *"simpan ke ../../rahasia.csv"* tidak bisa keluar dari folder ekspor. **Setiap tool harus melindungi dirinya sendiri**, bahkan di belakang persetujuan.
6. **Baris dibatasi untuk model, bukan untuk file.** `jalankan_query` menampilkan paling banyak 50 baris; tool ekspor menulis *semua* baris ke disk, setelah manusia berkata ya.

Pilihan Rancangan yang Perlu Diketahui

Tool skema vs skema di dalam prompt

	Tool skema (Contoh 2-5)	Skema di dalam prompt
Panggilan model	Panggilan tambahan untuk mencari	Lebih sedikit panggilan penjelajahan

	Tool skema (Contoh 2-5)	Skema di dalam prompt
Ukuran prompt	Kecil	Tumbuh mengikuti skema; dikirim pada **setiap** permintaan
Database besar	Berhasil (cari hanya yang perlu)	Terlalu besar
Freshness	Selalu terkini	Harus dibangun ulang kalau tabel berubah

Campuran keduanya juga sering dipakai: gambaran singkat tabel di prompt, ditambah `deskripsikan_tabel` untuk detailnya.

Memakai database lain

Tool-nya generik. `bangun_alat_sql(db_path)` membuat tool untuk file SQLite apa pun (latihan melakukan persis itu dengan database produksi harian). Untuk PostgreSQL atau MySQL, polanya sama:

1. Sambungkan dengan **pengguna khusus hanya-baca** (`GRANT SELECT` saja, pada tabel yang dibutuhkan)
2. Atur **statement timeout** pada pengguna atau koneksi itu
3. Pilih **replika baca** agar query analitik tidak memperlambat sistem yang hidup
4. Ganti `PRAGMA table_info` dengan katalog database (`information_schema`)
5. Utilitas `SQLDatabase` LangChain (di `langchain-community`) dan toolkit SQL membungkus tool serupa di atas SQLAlchemy kalau Anda lebih suka versi siap pakai

Daftar periksa produksi

- [ ] Akun database dengan `SELECT` saja, pada tabel yang dibutuhkan saja
- [ ] Timeout dan batas baris pada setiap query
- [ ] Data pribadi disembunyikan lewat redaction atau dikecualikan (pertimbangkan VIEW SQL yang menyembunyikan kolom sensitif)
- [ ] Persetujuan manusia untuk tool apa pun yang menulis, mengekspor, atau mengirim
- [ ] Batas panggilan model per permintaan
- [ ] Catat setiap pertanyaan, query SQL, dan ukuran hasil (audit log)
- [ ] Kumpulan uji berisi pertanyaan dengan jawaban yang diketahui (lihat latihan)

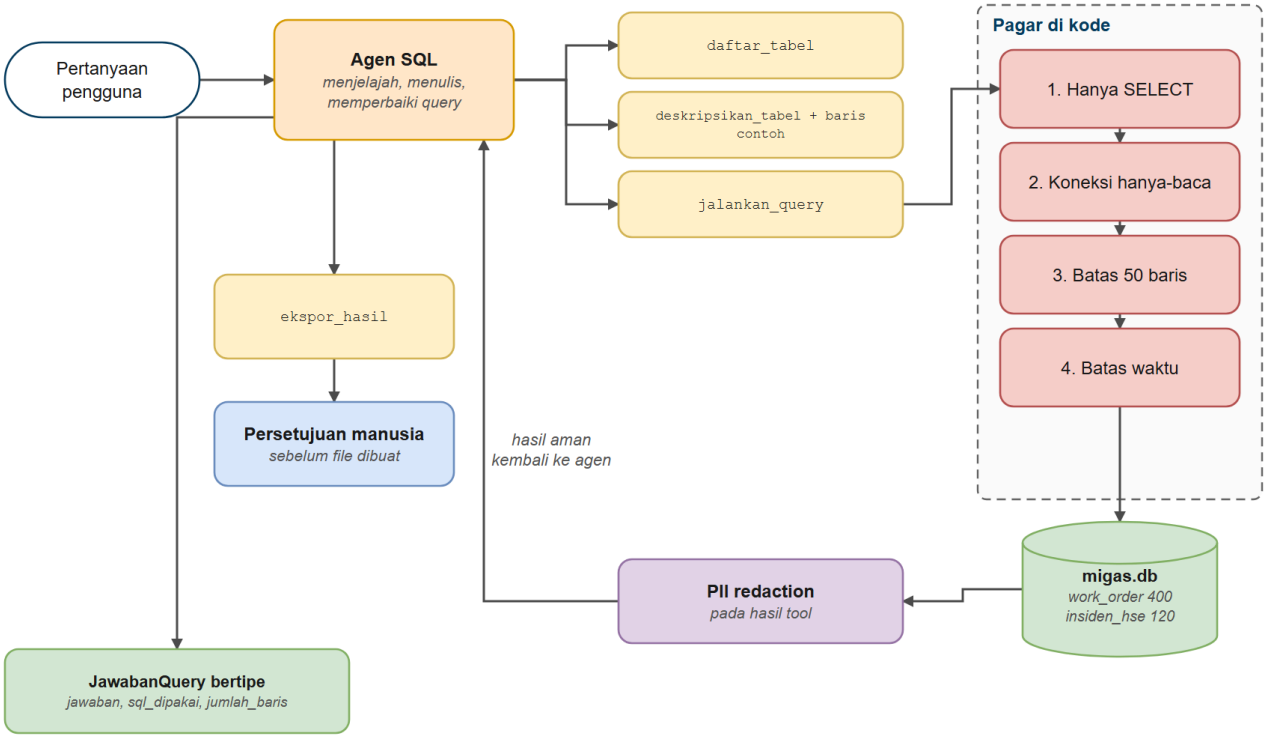
Poin Penting

- **Agen SQL menulis kode yang berjalan di atas data Anda**, jadi keamanan harus dipasang di kode dan izin akses, bukan di prompt
- **Biarkan agen melihat sebelum menulis**: `daftar_tabel` , `deskripsikan_tabel` dengan baris contoh, lalu `jalankan_query`
- **Jawaban salah yang tampak benar adalah kegagalan terburuk**: nilai contoh (`'H2S/Gas'` vs `'H2S'`) mencegahnya

- **Lapisi perlindungan Anda**: pemeriksaan hanya-SELECT (feedback ramah), koneksi hanya-baca (dinding sebenarnya), batas baris, dan timeout
- **Kembalikan error sebagai teks yang membantu**, bukan exception; selalu katakan kalau query tidak mengembalikan apa pun
- **Terapkan redaction pada data pribadi di hasil tool**, sebelum model bisa melihatnya
- **Membaca otomatis, menulis perlu persetujuan**
- **Uji terhadap kebenaran**: hitung jawaban benar secara independen lalu bandingkan
- **Polanya portabel**: `bangun_alat_sql(db_path)` bekerja untuk file SQLite apa pun; pakai ide yang sama dengan PostgreSQL

Peta Konsep

Bab ini mengajarkan cara membangun agen aman yang menjawab pertanyaan dari database:



Gambar 4. Alur agen SQL dengan pagar di kode

Latihan

Latihan bab ini berupa starter yang tinggal Anda isi: `latihan_1.py` dan seterusnya (atau `latihan_1.ipynb`), dengan bagian inti dikosongkan dan diberi komentar `TODO`. Petunjuk singkat ada di latihan.md; kode jawabannya ada di bagian **Kode Jawaban Latihan** pada PDF bab ini.

Latihan mencakup:

1. **Analis Produksi** - Terapkan pola pada database *baru* dan nilai agen Anda terhadap kebenaran yang dihitung dengan SQL
2. **Analis Percakapan** (Bonus) - Tambahkan memori agar pengguna bisa mengajukan follow-up question

Sumber Tambahan

- Tutorial Agen SQL LangChain - Panduan resmi dengan rancangan serupa
- SQLite: Database Hanya-Baca dan Nama File URI - Opsi `mode=ro`
- OWASP: Pencegahan SQL Injection - Mengapa akun dengan hak akses minimal penting

Apa Selanjutnya?

Kerja bagus! Anda sudah membangun agen yang bisa **menjawab pertanyaan dari database dengan aman**, memakai hampir semua yang dipelajari di beberapa bab terakhir.

Membangun di Atas Agen SQL

Tool yang Anda tulis sejauh ini dibuat oleh Anda, untuk satu program. Padahal banyak software sudah membuka data dan tindakannya lewat server standar. Menulis ulang konektor untuk setiap sistem (database, tiket, dokumen, kalender) akan tidak ada habisnya.

Di bab berikutnya Anda belajar **Model Context Protocol (MCP)**: cara standar bagi agen untuk menemukan dan memakai tool yang disediakan *program lain*, tanpa kode integrasi khusus. Setelah itu Anda menambahkan pencarian dokumen (embedding) dan RAG agentik.

Ide Proyek (Sejauh Ini)

Dengan yang sudah Anda pelajari, Anda bisa membangun:

- **Analis bad actor** - "Aset mana yang paling banyak menyumbang downtime bulan ini?"
- **Asisten statistik HSE** - Jawaban bertipe, persetujuan untuk ekspor, audit log setiap query
- **Analitik pemeliharaan** - Tanya tentang work order, downtime, dan suku cadang dari database tiket
- **Penjelajah data untuk karyawan baru** - Menjelaskan isi tiap tabel dengan contoh

Pemecahan Masalah

Masalah umum yang mungkin Anda temui saat membangun agen SQL:

"Agen menulis query untuk tabel yang tidak ada"

Penyebab: Ia tidak pernah melihat skema.

Perbaikan: Beri `daftar_tabel` dan `deskripsikan_tabel`, dan suruh di system prompt untuk memakainya *sebelum* menulis SQL:

```
system_prompt="SELALU mulai dengan daftar_tabel, lalu deskripsikan_tabel untuk tabel yang Anda perlukan, baru tulis query."
```

"Jawabannya 0 / 'tidak ada hasil' padahal saya tahu datanya ada"

Penyebab: Nilai filter tidak cocok dengan cara data disimpan (`'H2S'` vs `'H2S/Gas'`, huruf besar-kecil, spasi di ujung).

Perbaikan: Buat `deskripsikan_tabel` menampilkan baris contoh, dan suruh agen memeriksa nilai berbeda sebelum memfilter teks:

```
SELECT DISTINCT hazard FROM insiden_hse
```

"sqlite3.OperationalError: attempt to write a readonly database"

Penyebab: Sesuatu mencoba menulis lewat koneksi hanya-baca. Itu perlindungan yang sedang bekerja.

Perbaikan: Tidak ada yang perlu diperbaiki untuk query agen. Kalau *kode penyiapan Anda sendiri* perlu membuat tabel, pakai koneksi biasa (bukan `mode=ro`) hanya untuk langkah itu.

"sqlite3.OperationalError: unable to open database file"

Penyebab: Path salah, atau (untuk mode hanya-baca) filenya tidak ada. `mode=ro` tidak akan membuat file yang hilang.

Perbaikan: Periksa `DB_PATH`. Untuk `migas.db`, `sql_tools.py` membangunnya otomatis kalau belum ada; atau panggil `buat_database_migas()` sendiri.

"ModuleNotFoundError: No module named 'sql_tools'"

Penyebab: `sql_tools.py` diimpor dari folder yang sama dengan skrip. Itu berhasil saat skrip dijalankan langsung (`python hari4/bonus-agen-sql/code/02_schema_tools.py`), tetapi tidak dari notebook yang direktori kerjanya di tempat lain.

Perbaikan: Di notebook, tambahkan foldernya ke path lebih dulu:

```
import sys
sys.path.insert(0, "hari4/bonus-agen-sql/code")     # atau path absolut
```

"PermissionError: [WinError 32] ... being used by another process" (Windows)

Penyebab: Koneksi SQLite tidak pernah ditutup. Di Python, `with sqlite3.connect(...)` hanya melakukan commit atau rollback; ia **tidak** menutup koneksi, sehingga file tetap terkunci.

Perbaikan: Tutup secara eksplisit. Pembantu `koneksi_hanya_baca()` di `sql_tools.py` melakukannya, atau pakai `contextlib.closing`:

```
from contextlib import closing
with closing(sqlite3.connect(path)) as sambungan:
    ...
```

"Query mengembalikan terlalu banyak baris dan jawabannya terpotong"

Penyebab: Batas baris (50) melindungi jendela konteks model.

Perbaikan: Minta agregat ("10 teratas", "per area") atau tambahkan `LIMIT`. Untuk membuang seluruh isi, pakai tool ekspor yang dijaga persetujuan (Contoh 5).

"Filter kata kunci saya memblokir query yang tidak berbahaya"

Penyebab: Lapis 1 adalah pemeriksaan teks yang kasar. Teks seperti `WHERE deskripsi LIKE '%update%'` mengandung kata *update*.

Perbaikan: Terima positif palsu sesekali (model akan menulis ulang query-nya), atau pakai parser SQL sungguhan (misalnya paket `sqlparse`) di Lapis 1. Jangan pernah melemahkan Lapis 2.

Dependency

Pastikan paket yang dibutuhkan terpasang (tool SQL memakai `sqlite3` bawaan Python; sudah termasuk di `requirements.txt`):

```
pip install langchain langchain-openai langgraph python-dotenv
```

Bab ini juga memakai paket bersama `migas-shared` untuk membangun `migas.db`.

Referensi

Sumber ide dan teknik di bab ini:

- LangGraph. Build a custom SQL agent. Dokumentasi.
- Yu, T. et al. (2018). Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. Makalah.
- Pourreza, M. dan Rafiei, D. (2023). DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction. Makalah.
- OWASP. SQL Injection Prevention Cheat Sheet. Standar.
- SQLite. Uniform Resource Identifiers (mode=ro). Dokumentasi.
- Li, J. et al. (2023). Can LLM Already Serve as A Database Interface? A BIg Bench for Large-Scale Database Grounded Text-to-SQLs (BIRD). Makalah.

Kode Jawaban Latihan

Bagian ini memuat kode jawaban latihan bab bonus Agen SQL. Kerjakan dulu di file starter bab ini; buka bagian ini hanya untuk menyalin kode ke bagian `TODO` atau untuk mencocokkan hasil Anda. Setiap TODO bernomor `TODO(BQ.m)`, sama dengan nomor di file starter. Jalankan cell berurutan dari atas ke bawah.

Latihan 1: Analis Produksi

Buka `latihan_1.py` (atau `latihan_1.ipynb`) di folder bab ini. Tujuan: Menerapkan pola agen SQL pada database baru (produksi harian enam sumur) dan menilai agen terhadap kebenaran yang dihitung langsung dengan SQL. Ada 3 bagian yang harus Anda isi; kerjakan berurutan dari atas.

Langkah 2 - Kebenaran yang dihitung dengan SQL

Pada cell Langkah 2, ganti baris `raise NotImplementedError("TODO(BQ.1)")` di bawah komentar `TODO(BQ.1)` dengan kode berikut:

```
return {
    "jumlah_sumur": satu("SELECT COUNT(*) FROM Sumur"),
    "total_minyak_barel": satu(
        "SELECT SUM(Barel) FROM ProduksiHarian JOIN Produk USING (ProdukId) WHERE Produk.Nama =
'Minyak'"
    ),
    "sumur_air_terbanyak": satu(
        "SELECT Kode FROM ProduksiHarian JOIN Produk USING (ProdukId) JOIN Sumur USING
(SumurId) "
        "WHERE Produk.Nama = 'Air' GROUP BY Kode ORDER BY SUM(Barel) DESC LIMIT 1"
    ),
    "pendapatan_minyak_0610": satu(
        "SELECT SUM(Barel * HargaUSDPerBarel) FROM ProduksiHarian JOIN Produk USING (ProdukId)
"
        "WHERE Produk.Nama = 'Minyak' AND Tanggal = '2026-06-10'"
    ),
    "baris_produksi": satu("SELECT COUNT(*) FROM ProduksiHarian"),
}
```

Cell menjadi seperti berikut:

```
def kebenaran(db_path: Path) -> dict[str, str]:
    """Hitung jawaban benar langsung dengan SQL, tanpa agen."""
    with closing(sqlite3.connect(db_path)) as sambungan:
        def satu(sql: str) -> str:
            return str(sambungan.execute(sql).fetchone()[0])

        return {
            "jumlah_sumur": satu("SELECT COUNT(*) FROM Sumur"),
            "total_minyak_barel": satu(
                "SELECT SUM(Barel) FROM ProduksiHarian JOIN Produk USING (ProdukId) WHERE
Produk.Nama = 'Minyak'"
            ),
            "sumur_air_terbanyak": satu(
                "SELECT Kode FROM ProduksiHarian JOIN Produk USING (ProdukId) JOIN Sumur USING
(SumurId) "
                "WHERE Produk.Nama = 'Air' GROUP BY Kode ORDER BY SUM(Barel) DESC LIMIT 1"
            ),
            "pendapatan_minyak_0610": satu(
                "SELECT SUM(Barel * HargaUSDPerBarel) FROM ProduksiHarian JOIN Produk USING
(ProdukId) "
                "WHERE Produk.Nama = 'Minyak' AND Tanggal = '2026-06-10'"
            ),
            "baris_produksi": satu("SELECT COUNT(*) FROM ProduksiHarian"),
        }


def hanya_alfanumerik(teks: str) -> str:
    """Hapus pemisah ribuan dan spasi agar '1,234.5' dan '1.234,5' dibandingkan setara."""
    return "".join(karakter for karakter in teks if karakter.isalnum())
```

Langkah 3 - Database, tool SQL, dan agen

Pada cell Langkah 3, ganti baris `agent = None` di bawah komentar `TODO(BQ.2)` dengan kode berikut:

```
agent = create_agent(
    model,
    tools=[daftar_tabel, deskripsikan_tabel, jalankan_query],
    system_prompt=PROMPT_SISTEM_SQL + " Beri jawaban akhir sebagai JawabanQuery.",
    response_format=JawabanQuery,
    middleware=[ModelCallLimitMiddleware(run_limit=15, exit_behavior="end")],
)
```

Cell menjadi seperti berikut:

```
class JawabanQuery(BaseModel):
    """Jawaban akhir agen yang bertipe."""

    jawaban: str = Field(description="Jawaban dalam bahasa sehari-hari yang memuat angka atau
nama kuncinya")
    sql_dipakai: str = Field(description="Query SQL akhir yang dipakai ('tidak ada' bila tidak
ada query)")


folder_kerja = tempfile.TemporaryDirectory()
db_path = buat_database_produksi(Path(folder_kerja.name) / "produksi.db")
benar = kebenaran(db_path)

daftar_tabel, deskripsikan_tabel, jalankan_query = bangun_alat_sql(db_path)
agent = create_agent(
    model,
    tools=[daftar_tabel, deskripsikan_tabel, jalankan_query],
    system_prompt=PROMPT_SISTEM_SQL + " Beri jawaban akhir sebagai JawabanQuery.",
    response_format=JawabanQuery,
    middleware=[ModelCallLimitMiddleware(run_limit=15, exit_behavior="end")],
)
```

Langkah 4 - Fungsi pemeriksaan satu pertanyaan

Pada cell Langkah 4, ganti baris `cocok = False` di bawah komentar `TODO(BQ.3)` dengan kode berikut:

```
cocok = hanya_alfanumerik(diharapkan) in hanya_alfanumerik(jawaban)
```

Cell menjadi seperti berikut:

```
def periksa(pertanyaan: str, diharapkan: str) -> bool:
    response = agent.invoke({"messages": [HumanMessage(content=pertanyaan)]})
    hasil = response.get("structured_response")
    jawaban = hasil.jawaban if hasil else response["messages"][-1].content
    cocok = hanya_alfanumerik(diharapkan) in hanya_alfanumerik(jawaban)
    print(f"{'LOLOS' if cocok else 'GAGAL'}  {pertanyaan}")
    print(f"        diharapkan: {diharapkan}")
    print(f"        jawaban:    {jawaban[:120]}\n")
    return cocok


hasil_pemeriksaan = []
```

Latihan 2 (Bonus): Analis Percakapan

Buka `latihan_2.py` (atau `latihan_2.ipynb`) di folder bab ini. Tujuan: Menggabungkan agen SQL dengan memori agar pertanyaan lanjutan yang merujuk jawaban sebelumnya bisa dijawab. Ada 3 bagian yang harus Anda isi; kerjakan berurutan dari atas.

Langkah 2 - Agen dengan memori

Pada cell Langkah 2, ganti baris `agent = None` di bawah komentar `TODO(BQ.4)` dengan kode berikut:

```
agent = create_agent(
    model,
    tools=[daftar_tabel, deskripsikan_tabel, jalankan_query],
    system_prompt=PROMPT_SISTEM_SQL + " Pengguna boleh mengajukan pertanyaan lanjutan yang
merujuk jawaban sebelumnya.",
    middleware=[ModelCallLimitMiddleware(run_limit=15, exit_behavior="end")],
    checkpointer=InMemorySaver(),
)
```

Cell menjadi seperti berikut:

```
agent = create_agent(
    model,
    tools=[daftar_tabel, deskripsikan_tabel, jalankan_query],
    system_prompt=PROMPT_SISTEM_SQL + " Pengguna boleh mengajukan pertanyaan lanjutan yang
merujuk jawaban sebelumnya.",
    middleware=[ModelCallLimitMiddleware(run_limit=15, exit_behavior="end")],
    checkpointer=InMemorySaver(),
)
config = {"configurable": {"thread_id": "sesi-analis"}}
```

Langkah 3 - Pertanyaan pertama

Pada cell Langkah 3, ganti baris `response_1 = None` di bawah komentar `TODO(BQ.5)` dengan kode berikut:

```
response_1 = agent.invoke({"messages": [HumanMessage(content=pertanyaan_1)]}, config)
```

Cell menjadi seperti berikut:

```
pertanyaan_1 = "Sumur mana yang paling banyak memproduksi minyak secara total?"
print(f"Pengguna: {pertanyaan_1}")
response_1 = agent.invoke({"messages": [HumanMessage(content=pertanyaan_1)]}, config)
print(f"Agen: {response_1['messages'][-1].content}\n")
```

Langkah 6 - Memeriksa utas dan jawaban pembanding

Pada cell Langkah 6, ganti baris `status = None` di bawah komentar `TODO(BQ.6)` dengan kode berikut:

```
status = agent.get_state(config)
```

Cell menjadi seperti berikut:

```
def hitung_selisih(db_path: Path) -> str:
    """Hitung sendiri jawaban pertanyaan ke-3 dengan SQL, sebagai pembanding."""
    with closing(sqlite3.connect(db_path)) as sambungan:
        def pemenang(produk: str) -> int:
            return sambungan.execute(
                "SELECT SumurId FROM ProduksiHarian JOIN Produk USING (ProdukId) WHERE
Produk.Nama = ? "
                "GROUP BY SumurId ORDER BY SUM(Barel) DESC LIMIT 1", (produk,)
            ).fetchone()[0]

        def total(sumur_id: int) -> int:
            return sambungan.execute(
                "SELECT SUM(Barel) FROM ProduksiHarian WHERE SumurId = ?", (sumur_id,)
            ).fetchone()[0]

        return str(total(pemenang("Minyak")) - total(pemenang("Air")))


status = agent.get_state(config)
print(f"Utas menyimpan {len(status.values['messages'])} pesan (pertanyaan, panggilan SQL, dan
hasilnya).")
print(f"Jawaban benar pertanyaan ke-3, dihitung sendiri dengan SQL: {hitung_selisih(db_path)}")
```